COL1101 Lecture 17

2026/08/28

Topic: Priority Queues, Dictionaries, Binary Trees

Announcements

Exam OS testing in LH ~~325~~ 121

Recap: Priority Queues

- Enqueue
- Extract Min
  (min-first-out)

  (remove and return the min)
- is Empty, len, etc.

Items must be comparable:

$\leq$: relation on the items

Need: For all distinct items x, y: either $x \leq y$ or $y \leq x$.

i.e. $\leq$ must be a

total order

Implementations ?

	Enqueue	Extract Min
(Linked) List in sorted order	$\Theta(n)$	$\Theta(1)$
Unordered (Linked) List	$\Theta(1)$	$\Theta(n)$

Dictionary

Set of keys

Insert (key)

Delete (key)

Check membership (key)

(is Empty / len)

Implementations:

Sorted / Unsorted list.

Python Dictionary:

Set of key-value pairs.

Insert (key, value)

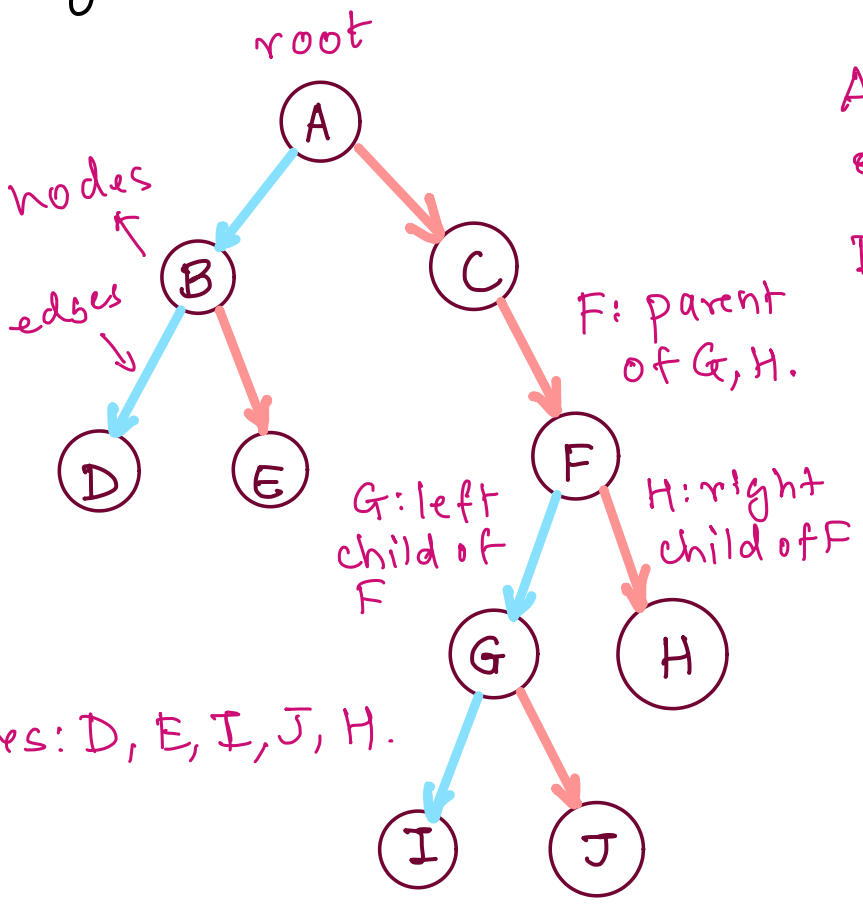
Delete (key, value) for a given key

Find value for a given key.

(is Empty / len)

Key must be immutable. (why?)

Binary Trees



F: parent of G, H.

G: left child of F

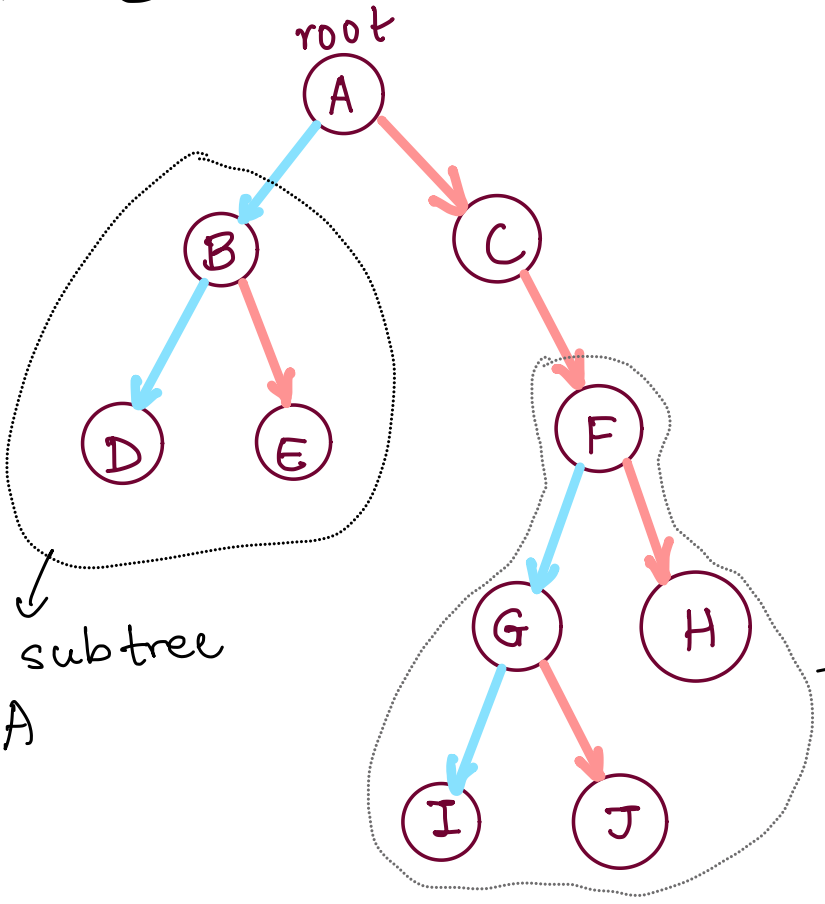
H: right child of F

Leaves: D, E, I, J, H.

Ancestors of F : F, C, A

Descendants of F : F, G, H, I, J

Subtrees

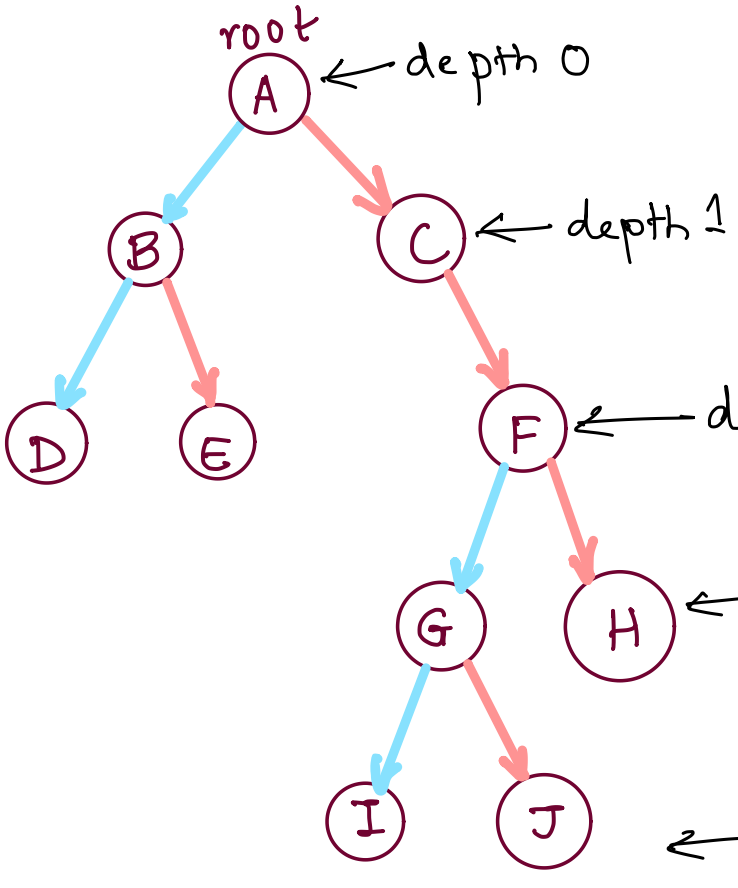


left subtree of A

→ subtree rooted at F.

= right subtree of C

Levels, Depth, Height



Depth of X: distance between root and X

Height of a tree: # nodes on a longest root-to-leaf path.

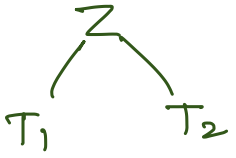
height = 1 + max depth of nodes

(0 if tree empty)

Inductive definition of binary trees:

Base case: Empty tree is a binary tree

Inductive case: If T_1, T_2 are binary trees and Z is a node, then



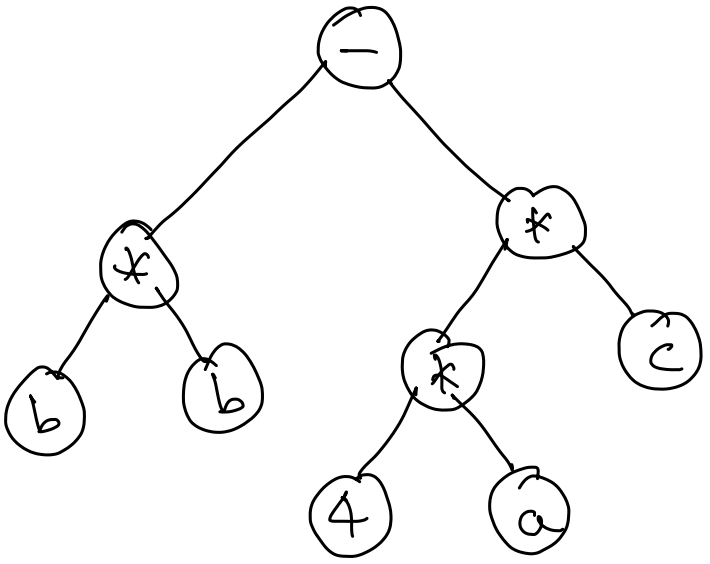
is a binary tree.

Exercise: Write inductive definitions of depth of node, height of tree, ancestor, descendant.

Cheat-sheet-worthy guideline:

Inductive definition ⟷ Recursive Algorithm / Program ⟷ Inductive proof of correctness

Expression Tree




```
evaluate (T):
    if T is a single node
        return T
    else # T = (op, T1, T2)
        v1 = evaluate (T1)
        v2 = evaluate (T2)
        return v1 op v2
```

Post-Order, Pre-Order, In-Order Traversals

```
postorder (T):
    if T not empty
        r ← root of T
        T1 ← left subtree of r
        T2 ← right subtree of r
        postorder (T1)
        postorder (T2)
        visit (r)
```

```
preorder (T):
    if T not empty
        r ← root of T
        T1 ← left subtree of r
        T2 ← right subtree of r
        visit (r)
        preorder (T1)
        preorder (T2)
```

```
inorder (T):
    if T not empty
        r ← root of T
        T1 ← left subtree of r
        T2 ← right subtree of r
        inorder (T1)
        visit (r)
        inorder (T2)
```